Sequence Dataset and Regularization Phase 2 Section Preview

Issue #74: Report Section
C-MAPSS Remaining Useful Life Study

I. Sequence Dataset and Regularization

The sequence experiments use the FD001 run-to-failure subset of NASA C-MAPSS [1]: 100 training engines and 20,631 observed cycles. The reference LSTM [2] has one layer of 64 units and reads a 30-cycle sensor window. This section describes the implemented data construction and the regularization study from issue #66. All reported scores are cross-validation (CV) scores on training engines. The official test cycles and RUL labels are not opened by the study; the frozen-model evaluation is reserved for issue #71.

A. Causal Windows, Stride and Padding

For engine i, let T_i be its final training cycle and $z_{i,t}$ its normalized sensor vector at cycle t. The window ending at t is

$$X_{i,t}[k,:] = z_{i,\max(1,t-L+k)}, \quad k = 1, \ldots, L, \tag{1}$$

with the first observed vector repeated on the left until the window has exactly L rows. Windows are built after sorting by engine and cycle; they contain neither future observations nor readings from another engine. Padding repeats the engine's own first reading, rather than zeros or the previous engine's last reading. The label belongs to the window's final cycle. Each PyTorch dataset item is a window–target pair; a batch has shape $[B, L, F]$, with $F = 15$ retained sensors for these folds.

Training stride s counts backwards from each engine's last cycle: $t \in \{T_i - ks : k \geq 0,\ T_i - ks \geq 1\}$. Thus the final cycle is always included, and repeated-first-cycle padding gives

$$N_s = \sum_i \lceil T_i / s \rceil \tag{2}$$

windows. Prediction and early-stopping validation always use $s = 1$, independently of the training stride. Every cycle can therefore be predicted, including the last cycle of an engine with fewer than L observations. At $s = 1$, padding preserves the first 29 windows of every FD001 training engine: without it, only 17,731 windows would remain, rather than 20,631.

Table I uses the same three engine-grouped folds (split seed 42) as Phase 1. Within each outer training partition, 20% of its engines are held out for early stopping, again with seed 42. Counts were checked against the actual window builder, not estimated by dividing the number of rows by the stride. Overlapping windows are not independent experimental units; the engine remains the unit of splitting.

TABLE I
Window counts for $L = 30$ with left padding. Fit, ES and CV denote fitting, inner early-stopping and outer scored engines. ES/CV use only $s = 1$; the all-training row is a reference count, not an extra study fit.

Partition	Engines	$s = 1$	$s = 2$	$s = 5$
All training	100	20,631	10,341	4,163
Fold 1 fit	52	10,588	5,308	2,134
Fold 1 ES	14	3,077	–	–
Fold 1 CV	34	6,966	–	–
Fold 2 fit	53	10,812	5,421	2,181
Fold 2 ES	14	3,167	–	–
Fold 2 CV	33	6,652	–	–
Fold 3 fit	53	10,647	5,336	2,151
Fold 3 ES	14	2,971	–	–
Fold 3 CV	33	7,013	–	–

B. Preprocessing Within Each Fold

The sensor list is selected from raw fitting-engine rows, before scaling. The default variability filter removes exactly constant channels; it does not discard a channel merely for having low variance. In all three fitting partitions it removes $s_1, s_5, s_{10}, s_{16}, s_{18}, s_{19}$ and retains 15 sensors, including the nearly constant s_6. The selected list is reused unchanged on that fold's ES and CV engines.

The regime normalizer learns operating-setting centroids and sensor means and population standard deviations from the fitting engines only. FD001 uses one regime. For retained sensor j in regime r,

$$z_{i,t,j} = \frac{x_{i,t,j} - \mu^{\text{fit}}_{r,j}}{\sigma^{\text{fit}}_{r,j}}. \tag{3}$$

A zero scale is replaced by one. Held-out engines use their nearest learned regime and its saved statistics; no held-out statistics are fitted. Engine identifier and target are metadata, and cycle index is not an input in this reference configuration. Operating settings determine the regime but are not direct LSTM inputs.

Fitting preprocessing before either engine split would leak held-out distributional information. Splitting individual windows would additionally place highly overlapping histories from the same engine on both sides. Here the fitting, ES and outer CV engine sets are disjoint. ES loss selects the retained training epoch [3]; outer CV engines are used only for scoring. Since model settings are selected using those CV scores, they remain exploratory tuning results, not an unbiased final generalization estimate.

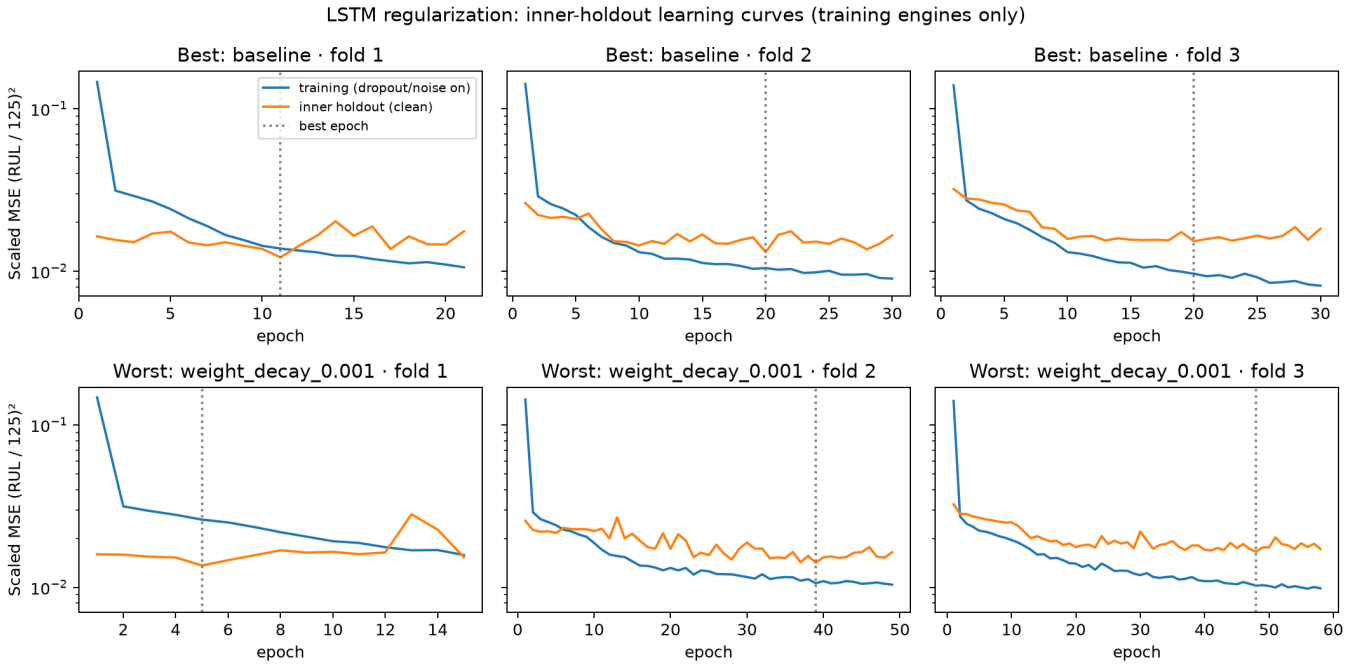


Fig. 1. Learning curves from #66 for the best capped setting (baseline) and worst (weight decay 10^{-3}), selected by outer CV selection RMSE. Each column shows one fold. Training is online minibatch MSE in training mode; the inner engine holdout is evaluated cleanly. Validation RMSE is squared and divided by 125^2 to match training-loss units. Vertical lines mark retained epochs; the logarithmic axis is shared. A continuing fall in training loss without a comparable holdout improvement is visible, but the tested alternatives do not improve outer CV scores.

C. Training Target and Selection Rule

The uncapped label is $y_{i,t} = T_i - t$. Capped training uses $y_{i,t}^{(C)} = \min(y_{i,t}, C)$, including the labels used by inner ES. Issue #55 will determine the training cap and selection threshold using training engines. Until that work lands, the study follows the existing LSTM configuration: provisional $C = 125$ and $K = 130$.

For outer fold f, let $A_f = \{(i,t) : i \text{ is scored in } f,\ y_{i,t} \leq K\}$. The selection score is

$$R_f = \sqrt{\frac{1}{|A_f|} \sum_{(i,t)\in A_f} (\widehat{y}_{i,t} - y_{i,t})^2}. \tag{4}$$

Outer scores always use true, uncapped labels. Table II reports the mean of the three R_f values and their sample standard deviation, rather than a pooled cycle-level RMSE. Uncapped training is shown beside each capped run. During optimization, targets are divided by C for capped runs, or by the largest fitting-engine label for uncapped runs; the latter scale is independent of window stride. Predictions are converted back to cycles before scoring.

These numerical choices are explicitly provisional. Saved out-of-fold predictions permit rescoring when #55 changes the selection threshold; a changed training cap requires retraining. The official test cannot be used to resolve either choice.

D. Regularization Study and Findings

The baseline has output dropout $p = 0.3$ [4], zero weight decay, no input noise, stride 1 and patience 10. With one LSTM layer, dropout acts on the final hidden state, not between recurrent layers. Each factor is varied separately; all other baseline settings are fixed. The shared trainer uses MSE, Adam [5] at 10^{-3}, batch size 256, gradient clipping at norm 1, a cosine learning-rate schedule over at most 100 epochs, CPU computation and training seed 42.

Gaussian input noise tests local perturbation robustness [6]: standard deviation is measured in fold-scaled sensor units, and new noise is drawn for each training batch using a dedicated seeded generator. Validation and prediction inputs remain clean, and the source arrays are unchanged. Increasing stride reduces both windows and optimizer updates per epoch; this is a sampling/compute comparison, not an equal-update experiment.

There are 11 settings, two target choices and three folds: 66 CPU fits, with approximately 14.8 minutes of fitting in total. None of the tested changes improves the baseline capped selection score of 15.78 ± 0.44. Weight decay 10^{-4} (15.83 ± 0.39) and noise 0.01 (15.87 ± 0.40) are close to it, rather than evidence of a gain. Patience 20 yields identical predictions to patience 10 for both targets but takes more training. Larger strides cost less fitting time and score worse. Noise lowers the uncapped score slightly, but this does not change selection under the predeclared capped-training criterion.

The recommendation is to retain dropout 0.3, weight decay 0, input noise 0, stride 1 and patience 10; no untested combination is proposed. Figure 1 shows the best/worst learning curves for every fold, without averaging histories of different lengths.

TABLE II

SELECTION RMSE (CYCLES), MEAN ± SAMPLE SD ACROSS THREE FOLDS, TRAINING SEED 42. EACH NON-BASELINE ROW CHANGES ONE FACTOR. CAPPED TRAINING USES $C = 125$; BOTH COLUMNS USE TRUE-RUL THRESHOLD $K = 130$. PROVISIONAL UNTIL #55.

Setting	Capped	Uncapped
Baseline	15.78 ± 0.44	30.79 ± 1.51
Dropout $p = 0.1$	16.81 ± 0.42	32.43 ± 2.05
Dropout $p = 0.5$	16.77 ± 1.05	30.84 ± 2.69
Weight decay 10^{-4}	15.83 ± 0.39	30.97 ± 3.21
Weight decay 10^{-3}	17.60 ± 1.00	32.76 ± 3.12
Noise $\sigma = 0.01$	15.87 ± 0.40	30.13 ± 1.11
Noise $\sigma = 0.05$	16.04 ± 0.43	30.06 ± 1.64
Stride $s = 2$	17.47 ± 1.03	32.33 ± 3.98
Stride $s = 5$	17.52 ± 0.61	31.62 ± 2.44
Patience 5	16.25 ± 0.92	30.79 ± 1.51
Patience 20	15.78 ± 0.44	30.79 ± 1.51

The SD describes fold variation, not training-seed uncertainty or a confidence interval. Only one training seed was tested, and the baseline was already tuned on these folds. This null result therefore applies to the tested settings and current provisional rule; additional seeds and the final #55 rule must precede any final freeze recommendation.

REFERENCES

[1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *2008 International Conference on Prognostics and Health Management*. IEEE, 2008, pp. 1–9.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] L. Prechelt, "Early stopping—but when?" in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science, G. B. Orr and K.-R. Müller, Eds. Berlin, Heidelberg: Springer, 1998, vol. 1524, pp. 55–69.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html

[5] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[6] C. M. Bishop, "Training with noise is equivalent to Tikhonov regularization," *Neural Computation*, vol. 7, no. 1, pp. 108–116, 1995. [Online]. Available: https://www.microsoft.com/en-us/research/publication/training-with-noise-is-equivalent-to-tikhonov-regularization/